PRECISION CARE CHALLENGE 2026 · JUDGE Q&A DEFENSE

# NeuroPilot

Every question a judge can ask — answered from the build. Anchored by the live question: **"Why are patients with the same score prioritized differently?"** — with the five-layer answer, the explicit tiebreaker policy, and the honest concession.

| | | |
|---|---|---|
| RISK MODEL<br>**AUC 0.842 ± 0.032 (CV)** | PROGRESSION FORECAST<br>**AUC 0.770 · MAE 0.61 pts** | COHORT<br>**800 subjects · 4 stages** |
| SCOPE GUARD<br>**Tier · reasoning · next test** | TESTS<br>**24 pytest · E2E verified** | DEPLOYED<br>**Railway · push-to-deploy** |

All figures verified against `artifacts/eval_report.txt`, `artifacts/progression_report.txt` and the deployed build. Prepared 2026-09-10.

# Contents

# 1 The Question You Were Asked

> "Why are patients with the same score prioritized differently?"

## 1.1 The answer in one sentence

They almost never actually have the same score — the dashboard **displays** scores rounded to two decimals (`0.81` can be 0.8067 or 0.8132), while **ranking and all decisions use full-precision model output**; and when two patients are clinically indistinguishable, the system deliberately surfaces *why* each one is ranked there (SHAP attribution) instead of silently inventing a preference — that decision stays with the clinician.

## 1.2 The five-layer answer (say it in this order)

**1. Display rounding, not decision rounding.** The score is a continuous float64 from a gradient-boosted ensemble. The UI shows `toFixed(2)` for readability; the ranked table sorts by the full-precision value. Two rows reading "0.96" are almost never equal internally — the tie is an artifact of the display, and the ordering between them is real.

**2. Priority is not the score alone — it is (tier, stage).** The rule engine gates the next test on the *risk tier* (high > 0.7 · medium ≥ 0.4 · low) crossed with the *pipeline stage* the patient has reached. A 0.65 at Stage 1 (medium) gets a blood panel; a 0.65 at Stage 3 (medium) gets a 12-month follow-up MRI. Same displayed score, different indicated action — by clinical design.

**3. Tier boundaries change behavior at the same "displayed" score.** 0.69 vs 0.71 both display near "0.7" but straddle the high-tier threshold — at Stage 3 that is the difference between a follow-up MRI and an amyloid/tau PET. The thresholds are configurable (env-overridable) and shown on the gauge.

**4. Same score, different reasoning — and we show it.** Two patients at 0.75 can arrive there by different paths: one via cognition (MMSE 20, declining), another via biomarkers (p-tau 6.1, hippocampus 2.1 cm³) with a decent MMSE. SHAP attribution exposes exactly this. If a clinician must break a near-tie, they break it on *evidence composition*, which the UI makes visible per patient.

**5. In autonomous mode the choice is deterministic and auditable.** `workup/next` picks the highest-scoring subject whose tier still indicates a test (exact code: `max(candidates, key=score)`). Because features are continuous, exact float ties across 800 subjects are measure-zero; the audit trail logs every pick with before/after rank.

## 1.3 If we wanted an explicit tiebreaker (production answer)

We would add a deterministic, documented chain — never a hidden one: ① **score (full precision)** → ② **higher conversion probability** (we already compute a 12-month progression probability per patient — the natural clinical tiebreaker) → ③ **longer waiting time** → ④ **stable patient ID**. The key design principle: ties are resolved by *declared, auditable policy*, not by silent side effects of sort stability.

## 1.4 The honest concession (judges reward this)

"If two patients are truly indistinguishable on every measured dimension, the system **refuses to fake a preference** — it presents both reasonings side by side and leaves the call to the clinician. In a clinical decision-support product, an unexplainable ordering is worse than a visible tie." Then note the roadmap item: surface the explicit tiebreaker policy in the UI so the ordering is always explainable end-to-end.

# 2 Problem & Product

## 2.1 What problem does NeuroPilot solve?

Memory-clinic and trial-screening workflows drown in undifferentiated referral lists: every patient waits the same, test ordering is ad-hoc, and the patients who would benefit most from early workup are found late. NeuroPilot turns a static cohort into a **continuously re-ranked, self-advancing priority queue**: each patient carries a model score, an explanation, a pipeline position (Cognitive → Blood → MRI → PET), and a 12-month outlook — and the system itself can drive the workup, one indicated test at a time, re-scoring and re-ranking as results land.

## 2.2 Why prioritization, not diagnosis?

Because that is the honest, safe, and regulated scope for this evidence base. A diagnosis changes a life; our models are trained on synthetic, ADNI-shaped data and would need local validation before any clinical use. So the system outputs exactly three things — **risk tier, reasoning, recommended next test** — and no endpoint or screen contains a diagnosis field. It makes the queue smarter; it does not make the call.

## 2.3 Who is the user, and what do they see?

A clinician or trial coordinator. Overview: tier distribution, risk histogram, stage funnel, top-8 shortlist. Patients: the full ranked, filterable table. Detail: score gauge with thresholds, all 10 SHAP factors grouped by stage, pipeline stepper, recommended next step, interactive results entry, timestamped audit trail. One click: the full-page 12-month progression forecast.

## 2.4 Why four stages, in that order?

It mirrors the real diagnostic funnel and a cost/invaiseness gradient: **cognition** (cheap, universal, always measured) → **blood biomarkers** (cheap draw, p-tau181 + Aβ42/40) → **MRI volumetrics** (hippocampal atrophy) → **PET** (amyloid/tau — most expensive, most invasive, ordered only when the tier still indicates it). Each stage refines the score; the updated tier decides whether the next stage is even indicated. Low-tier patients stop early — that is the resource-saving point.

## 2.5 How is this different from "just an ML model"?

The model is one component of five: a **deterministic rule engine** decides what tests are indicated (auditable if/else, not another ML layer), an **explanation layer** (SHAP) makes every score defensible, a **pipeline/audit layer** tracks every order, result and re-score, an **autonomy layer** drives the cohort workup under tier gating, and a **forecasting family** projects 12-month trajectories. Remove the model and the product still has a defensible clinical workflow — that is deliberate.

## 2.6 Walk me through the product in 30 seconds.

Open the dashboard: 800 ranked subjects. Toggle **Autonomous Triage**: the system picks the highest-priority subject itself, orders its next indicated test, derives the result, re-scores with the trained model, and re-ranks the queue — the banner shows `SYN-0087 · BLOOD abnormal · 0.68 → 0.81 (high) · #12 → #2`. Open any patient: gauge, stage-grouped SHAP, stepper, audit trail, and a Progression Probability button that opens the full 12-month forecast chart.

# 3 Data

## 3.1 Real data or synthetic?

Both, by design. **Real:** public OASIS-1 longitudinal (150 subjects, 373 visits) runs the full ETL — dedupe, missing-value audit (SES ×19, MMSE ×2 flagged, never silently dropped). **Synthetic:** an 800-subject ADNI-1-proportioned cohort (200 CN / 400 MCI / 200 AD) with severity-correlated measures at all four stages — this is the served default because OASIS-1 has *no blood, MRI-volumetry or PET columns*, and a 4-stage product needs 4-stage data.

## 3.2 Why not use real ADNI data?

ADNI requires registration, data-use certification and manual downloads — not redistributable in a hackathon repo. We built the generator to ADNI-1's published composition and biomarker ranges specifically so the **swap is one command**: point `train_model.py --data` at a real cohort and the same pipeline, guardrails and API serve it unchanged.

## 3.3 Is the synthetic cohort clinically plausible — or did you hand the model the answer?

It is generator-shaped, and we say so plainly: measures are severity-correlated with realistic reference ranges (p-tau 0.6–7.5 pg/mL, Aβ42/40 0.04–0.16, hippocampus 1.6–4.3 $cm^3$), CDR included only as the label source and excluded from features. The honest framing: the model demonstrates the *methodology* end-to-end; performance numbers on synthetic data are a floor, not a claim. The model card, /model/info and /health all disclose `data_mode: synthetic`.

## 3.4 How is missing data handled?

Two distinct cases. **Un-ordered tests** stay NaN — XGBoost consumes missing values natively, and SHAP attributes the learned missing-value path (shown in the UI as a dimmed `model default` badge, so un-measured stages are visible, not hidden). **Measured-but-missing values** (e.g. OASIS SES/MMSE gaps) are median-imputed inside the sklearn Pipeline, which is bundled into the served artifact — imputation is therefore identical at train and serve time.

## 3.5 What prevents subject overlap between train and test?

Subject-level splitting: a patient is entirely in train or entirely in test — no visit-level leakage. Stratified by label; 80/20. The same discipline carries into the progression models (stratified by conversion).

## 3.6 What is your reproducibility story?

Seeded generators (fixed seed in both cohort scripts), deterministic follow-up simulation, versioned artifacts with `trained_at` and full metric cards (`artifacts/eval_report.txt`, `progression_report.txt`), and committed artifacts so a fresh deploy serves the exact model that was evaluated.

# 4 Model — Risk Scoring

## 4.1 Why XGBoost?

Tabular, mixed-type, small-to-medium clinical data is its strength: it handles missing values natively (essential for staged workups), captures non-linear threshold effects (MMSE 24 vs 23 is not a linear event), and trains in seconds so re-scoring on every result is practical. A RandomForest fallback ships alongside (test AUC 0.874 vs 0.871) — and the baseline gap proves the complexity is warranted.

## 4.2 Why not deep learning / why not an LLM?

With 800 subjects, a deep net would overfit and lose the SHAP-speed explainability we serve per request. LLMs are the wrong tool for a calibrated numeric risk estimate — we use deterministic code where determinism matters and reserve narrative language for the UI copy.

## 4.3 Exactly what are the features, and why is CDR excluded?

Ten, spanning all four stages: `mmse`, `mmse_change`, `age`, `education_years`, `sex`, `ptau181`, `abeta4240`, `hippocampal_volume`, `amyloid_positive`, `tau_positive`. CDR is excluded because it is a **clinician rating that effectively encodes the diagnosis** — training on it is label leakage and the model would be a parrot, not a predictor.

## 4.4 What are the actual numbers?

5-fold CV AUC **0.842 ± 0.032** (the honest estimate), held-out test AUC 0.871, accuracy@0.5 **0.750** on 160 unseen subjects, majority-class baseline 0.581 — about **+17 points over naive guessing**. Confusion matrix [[52 15],[25 68]]. Per-class: high-tier precision 0.819 / recall 0.731.

## 4.5 Is AUC 0.84 good enough for clinical use?

For *prioritization*, yes-in-principle; for deployment, not yet — because the data is synthetic. Rank quality is what the product needs (AUC measures exactly that), and 0.84 with a 0.58 baseline is a solid signal. The caveat is stated in the model card, the eval report and the UI footer: synthetic training data, optimistic holdout (early stopping used the test set), real deployment requires local validation.

## 4.6 Where do the thresholds 0.7 / 0.4 come from?

They are policy, not model output — deliberately configurable via environment (`HIGH_RISK_THRESHOLD` / `MEDIUM_RISK_THRESHOLD`) so a clinical site can calibrate to its own capacity and prevalence. The gauge shows them explicitly; nothing about tiering is hidden inside the model.

## 4.7 How does a score change when a test result lands?

The result is written into the patient record, the 10-feature vector is recomputed (NaN → measured value), and the served pipeline re-scores immediately — score and tier update live, and the audit trail timestamps the transition. In the verification run an abnormal blood panel moved one subject **0.68 → 0.81** and its rank **#12 → #2**.

# 5 Explainability

## 5.1 Why SHAP?

It is the standard for tree-ensemble attribution with solid theoretical grounding (Shapley values): consistent, locally accurate, and fast via TreeExplainer. It gives both views the product needs — **global** importance (which features drive the cohort: MMSE 1.068, p-tau181 0.304, Aβ42/40 0.233, age 0.184, hippocampal volume 0.175) and **per-patient** reason codes with signed contributions.

## 5.2 What is the "model default" badge?

The contribution shown for a **test that was never ordered** — XGBoost routed that NaN down a learned default path. We badge it (dimmed, with a footnote) because presenting a number for an un-measured biomarker would be misleading. The moment the test completes, the measured contribution replaces it. This is a transparency feature judges consistently notice.

## 5.3 Can a clinician challenge the score?

Yes — that is the design. Every factor is shown with its measured value and signed contribution, grouped by pipeline stage; the clinician can see the score is "driven by cognition, biomarkers normal" and act on judgment. The rule engine additionally refuses to auto-escalate routine follow-ups (409) without an explicit `override: true`, which is logged.

# 6 Progression Forecaster

### 6.1 What exactly does "probability of clinical progression" mean?

The model's estimate that the patient crosses into the next diagnostic phase within **12 months**: CN → MCI, MCI → AD, or AD → severe decline (MMSE < 15). It is trained on the same 10-feature baseline vector, with labels from simulated follow-up trajectories driven by published base rates (CN ~4%/yr, MCI ~12%/yr, AD ~28%/yr) modulated by each patient's biomarker evidence.

### 6.2 How accurate is the forecast?

MMSE-delta regressor: test MAE **0.611 points** (mean-baseline 1.043) — a year-ahead forecast within ~0.6 points; $R^2$ 0.679. Conversion classifier: ROC AUC **0.770**, PR AUC 0.465 against 16.6% prevalence (**~2.8× lift**), Brier 0.143. The uncertainty is visible: the trajectory chart shows a ± band (~75% interval) around the predicted path.

### 6.3 How is the "projected tier" computed? Is it a third model?

No third model — deliberately. The **current risk model** is re-scored on the projected future vector (age+1, forecast MMSE, carried-forward biomarkers). One consistent model family end-to-end means the "today vs 12-month" comparison on screen is apples-to-apples.

### 6.4 What are the stage-score markers on the trajectory chart?

The risk score **at the moment each stage test was completed**: the trained model is re-scored on the patient's data *as it existed then* (later-stage results masked to unmeasured). You literally watch the score climb as blood → MRI → PET results land — the chart tells the same story the live system performed.

### 6.5 Is the forecast a diagnosis or a guarantee?

Neither — the card carries the disclaimer in the UI and the API payload. It is a "nothing changes" projection under standard care; an intervention (e.g. anti-amyloid therapy) would alter the trajectory. And it is trained on simulated trajectories shaped by published dynamics — production would retrain on a real longitudinal cohort.

# 7 Rule Engine & Pipeline Logic

## 7.1 Why a rule engine for escalation instead of another ML model?

Because test-ordering policy must be **auditable, defensible and inspectable** — a transparent (stage × tier) table beats a second black box deciding who gets an MRI. In a clinical/regulatory context, "the if-statement did it" is a feature. The ML decides *how risky*; the rules decide *what that risk means procedurally*.

## 7.2 State the escalation matrix.

| Stage | Low | Medium | High |
|---|---|---|---|
| **1 Cognitive** | Re-screen 12 mo | Blood panel | Blood panel |
| **2 Blood** | Re-screen 12 mo | MRI volumetrics | MRI volumetrics |
| **3 MRI** | Return to screening | Follow-up MRI 12 mo | Amyloid/tau PET |
| **4 PET** | — pathway complete → multi-disciplinary review — | | |

## 7.3 How is the clinician kept in the loop?

Three mechanisms: (1) "Schedule/Return" recommendations are never auto-executed — `advance-stage` 409s unless `override: true`; (2) every order, result, re-score and override is timestamped in the reasoning trail (mirrored to Postgres); (3) the autonomous loop is opt-in and can be toggled off mid-run — the system stops, mid-state intact.

## 7.4 What happens at Stage 4 / pathway end?

The pipeline reports complete and routes to multi-disciplinary review. In autonomous mode that subject simply drops out of the testable queue — the system never loops tests for completeness's sake.

# 8 Autonomy & Safety

## 8.1 How autonomous is it, really?

Fully, within a hard scope: `workup/next` ranks the cohort, picks the top subject whose tier indicates a test, orders it, derives a plausible result, re-scores, re-ranks — and reports before/after rank for audit. `workup/run` batches up to N steps. It stops cleanly when no tier indicates a test. It never prescribes treatment, never diagnoses, and never escalates routine follow-ups without override.

## 8.2 What stops it ordering unnecessary tests?

The tier gate: a low tier at any stage returns "re-screen in 12 months" — no test indicated, the cascade stops with a 409 and a stated reason. Cost escalates with stage by design (PET only for Stage-3 high tier), so the expensive modalities are naturally rationed by the same table.

## 8.3 What if the model is wrong?

Bounded blast radius: a wrong score mis-*orders* the queue — it does not diagnose or treat. The clinician sees the attribution behind every score and can override anything; every automated action is reversible and logged. And the failure mode we care about (missed high-risk patients) is measurable: recall on the held-out set is reported in the eval report rather than buried.

## 8.4 Are the "results" real?

No — ordering a test derives a clinically plausible outcome from the subject's severity profile. That is a declared demo stand-in for a real LIS/RIS integration, and it is what makes the live loop (order → result → re-score → re-rank) demonstrable without a hospital backend.

# 9 Engineering & Architecture

### 9.1 Draw the system.

**Scripts** (Python 3.11: generate/ingest/simulate/train) → **artifacts** (pipeline.joblib + progression models + model cards, committed) → **FastAPI** (loads cohort, re-scores everyone at startup via vectorized SHAP batch; rule engine; progression serving; optional Postgres persistence) → **React + Vite + Tailwind** SPA (pure API client, zero mock data). Single Dockerfile: builds the UI, serves it from the API process on $PORT.

### 9.2 How is the model served? Any latency concerns?

In-process — the sklearn Pipeline + XGBoost joblib loads at startup with a cached SHAP TreeExplainer; per-request scoring is milliseconds and startup re-scores all 800 subjects in one vectorized batch. At this scale a separate model server (MLflow/TorchServe) would be over-engineering; the boundary where that changes is clearly understood.

### 9.3 Why FastAPI / React / Postgres?

FastAPI: async, Pydantic-validated contracts (which structurally enforce "no diagnosis field"), auto Swagger for judges to poke endpoints live. React+Vite+Tailwind: fast, componentized, product-grade UI with zero hardcoded data — `src/api.js` is the only data source, so an API outage shows an error, never fake rows. Postgres (optional, active on Railway): relational integrity fits one-row-per-event clinical history; the schema mirrors the blueprint (patients, cognitive_assessments, lab_results, pipeline_history…).

### 9.4 What is tested?

24 pytest cases covering the API contract end-to-end (sorting, filtering, tiering, 404/409/422 semantics, workup loop, progression) plus DB seed foreign-key integrity. The frontend was verified with headless-Chrome E2E probes (navigation round-trips, chart rendering, autonomous banner).

### 9.5 How do you handle concurrent edits / race conditions in the demo?

Single-writer in-process state for the demo scope; Postgres persistence when configured, with every state change event-logged. Production would move to optimistic locking on patient records — acknowledged as out of demo scope.

# 10 Deployment

## 10.1 Where is this deployed and how?

Railway, from a single Dockerfile: stage 1 builds the React bundle, stage 2 is a Python 3.11 slim runtime that copies the backend, the committed artifacts, and the built SPA, and serves everything from one uvicorn process on $PORT. Push-to-deploy from GitHub. Postgres via Railway plugin with `DATABASE_URL=${{Postgres.DATABASE_PRIVATE_URL}}`.

## 10.2 Why are model artifacts committed to git? Isn't that an anti-pattern?

It is a deliberate deployment trade-off: committed artifacts mean a fresh container serves the exact evaluated model with **zero training at boot** — critical for a judged demo's reliability. Datasets are *not* committed (licensing + reproducibility: the cohorts regenerate from seeded scripts in seconds). Production CI would swap this for artifact registry + version pinning; the eval reports already pin what each artifact should produce.

## 10.3 Scaling & security posture?

Scaling: the app is stateless per request (state lives in the DB when enabled) — scale horizontally behind a load balancer; model inference is cheap. Security/PHI: this demo holds synthetic subjects only; production would need PHI controls (encryption at rest, access control, audit retention) and is explicitly listed out of scope — we would not ship it otherwise.

# 11 Limitations & Roadmap

## 11.1 State the limitations — before we find them.

We lead with them: (1) **synthetic training data** — methodology demo, not a clinical claim; disclosed in the model card, /model/info, /health. (2) **Simulated progression labels** shaped by published dynamics, not observed outcomes. (3) **Simulated test results** stand in for LIS/RIS. (4) **Optimistic holdout** — early stopping used the test set; CV AUC is the honest number. (5) **PET signal thin** (amyloid/tau correlate in the generator). (6) **No auth/multi-clinician**, demo prototype. Every one of these has a named production answer.

## 11.2 What would productionization actually take?

Retrain on a real registered cohort (ADNI/OASIS-3) with local validation; replace derived results with LIS/RIS integration; auth + role-based access; artifact registry and CI; drift monitoring (score distribution + SHAP stability dashboards); prospective evaluation of the escalation matrix with clinical governance. The architecture is already shaped for this swap — that was the design constraint.

## 11.3 What would you build next?

The explicit tiebreaker policy surfaced in the UI (score → conversion probability → waiting time → ID); intervention-aware forecasting ("what if anti-amyloid therapy starts now"); cohort-level capacity planning (queue vs scanner slots); and FHIR interoperability so orders/results speak hospital language natively.